

Innovus Tutorial

May 2026



NORTHWESTERN
UNIVERSITY

McCormick

Northwestern Engineering

Innovus Tutorial

Before going to next steps, please note that those lines that start with ‘#’ are explanation, lines that follow with ‘\$’ are commands and you need to copy and then paste in your terminal and press enter.

- 1) # Type command: “source /vol/ece303/genus_tutorial/cadence.env”.

```
$ source /vol/ece303/genus_tutorial/cadence.env
```

- 2) # You will follow the example in /vol/ece303/backend_tutorial

You need to finish the synthesis job first:

Create “Frontend” folder and copy all the files from /vol/ece303/backend_tutorial/Frontend into the folder. Follow the README file and run through synthesis to generate all the files.

Create “Backend” folder in the same directory as “Frontend” folder and go into this folder.

```
$ mkdir Backend
```

```
$ cd Backend
```

- 3) # Add “VDD” and “VSS” into the file in :

../Frontend/Synthesis/synth_pipe_high/float16_multiplier_syn.v as below and save the file:

- a. **Open** float16_multiplier_syn.v and add “VDD” and “VSS”:

```
$ vim ../Frontend/Synthesis/synth_pipe_high/float16_multiplier_syn.v
```

```
module float16_multiplier(clk, rst_n, float_a_i, float_b_i,
    float_product_o,VDD,VSS);
    input clk, rst_n;
    input [15:0] float_a_i, float_b_i;
    output [15:0] float_product_o;
    inout VDD, VSS;
```

#After modification, quit float16_multiplier_syn.v file

```
$ :wq
```

Note we use “vim” text editor in this example. You can use other text editor, e.g. emacs

- 4) # Copy float16_malu_conv.view to this folder.

Note: your sdc file path might be different, remember to modify .view file accordingly.

```
$ cp /vol/ece303/backend_tutorial/Backend/float16_multiplier.view .
```

\$ vim float16_multiplier.view and fill in your sdc file path generated during Synthesis. You need to change the file path if it is different from below.

```
create_constraint_mode -name sdc -sdc_files {../Frontend/Synthesis/synth_pipe_high/float16_multiplier_syn.sdc}
```

#After modification, quit the float16_multiplier.view file

```
$ :wq
```

- 5) # Open innovus

```
$ innovus
```

Your terminal should show something similar to this and innovus window should pop up on the side:

```

[ylv0377@joker Frontend]$ innovus
[ylv0377@joker Backend]$ innovus
WARNING: HOST <joker.ece.northwestern.edu> DOES NOT APPEAR TO BE A CADENCE SUPPORTED LINUX CONFIGURATION.
For More Info, Please Run '<cdsroot>/tools.lnx86/bin/checkSysConf' <productId>.

Cadence Innovus(TM) Implementation System.
Copyright 2016 Cadence Design Systems, Inc. All rights reserved worldwide.

Version:          v16.20-p002_1, built Tue Nov 8 11:31:23 PST 2016
Options:
Date:             Tue May 19 00:00:00 2026
Host:             joker.ece.northwestern.edu (x86_64 w/Linux 4.18.0-553.124.1.el8_10.x86_64) (8cores*8cpus*Intel(R) Cor
e(TM) i7-9700 CPU @ 3.00GHz 12288KB)
OS:              Red Hat Enterprise Linux release 8.10 (Ootpa)

License:
      invs      Innovus Implementation System      16.2      checkout succeeded
      8 CPU jobs allowed with the current license(s). Use setMultiCpuUsage to set your required CPU count.
Create and set the environment variable TMPDIR to /tmp/innovus_temp_4134731_joker.ece.northwestern.edu_ylv0377_HWrSN3
.

**INFO:  MMMC transition support version v31-84

innovus 1> █

```

6) # Import Design in GUI (File -> Import Design)

- a. **Verilog:** use the gate level netlist, which is generated in genus (../Frontend/Synthesis/synth_pipe_high/float16_multiplier_syn.v) with **Top Cell** by clicking **Auto Assign**
- b. **LEF Files:** “/vol/ece303/genus_tutorial/NangateOpenCellLibrary.lef”
- c. **MMMC View Definition File:** “float16_multiplier.view”
- d. **Power:**
 - i. Power Nets: VDD
 - ii. Ground Nets: VSS
- e. Click **OK**

Make sure the Verilog file and View file exist and have the correct path typed in the form, otherwise, you cannot finish import. You can also give the absolute path of the files.

Design Import

Netlist:

☒ Verilog
Files: ../Frontend/Synthesis/synth_pipe_high/float16_multiplier_syn.v ...
Top Cell: ☒ Auto Assign ☐ By User: ...

☐ OA
Library: ...
Cell: ...
View: ...

Technology/Physical Libraries:

☐ OA
Reference Libraries: ...
Abstract View Names: ...
Layout View Names: ...

☒ LEF Files
/vol/ece303/genus_tutorial/NangateOpenCellLibrary.lef ...

Floorplan

IO Assignment File: ...

Power

Power Nets: VDD
Ground Nets: VSS
CPF File: ...

Analysis Configuration

MMMC View Definition File: float16_multiplier.view ...
Create Analysis Configuration ...

OK Save... Load... Cancel Help

- 7) #commitConfig. **For the following lines with “\$”, you should copy, and then paste into the Terminal, which means you will not use the GUI to operate any more.**

```
$ setDesignMode -process 45
$ fit
$ setDrawView fplan
$ getIoFlowFlag
```

- 8) # This command prepares your floor plan layout and assign its option. You can find the details in the GUI.

```
$ floorPlan -r 1.0 0.63 2 2 2 2
$ uiSetTool select
$ getIoFlowFlag
```

- 9) # the following commands set your VDD and VSS pins.

```
$ globalNetConnect VDD -type pgpin -pin VDD -inst *
$ globalNetConnect VSS -type pgpin -pin VSS -inst *
$ globalNetConnect VDD -type tiehi
$ globalNetConnect VSS -type tielo
```

```
innovus 8> globalNetConnect VDD -type pgpin -pin VDD -inst *
innovus 9> globalNetConnect VSS -type pgpin -pin VSS -inst *
innovus 10> globalNetConnect VDD -type tiehi
innovus 11> globalNetConnect VSS -type tielo
innovus 12> █
```

10) # **Edit Pin Locations** (Edit -> Pin Editor) **(Use GUI for this step)**

For example, you can allocate the output signals 'float_product_o[]' by setting bellow,

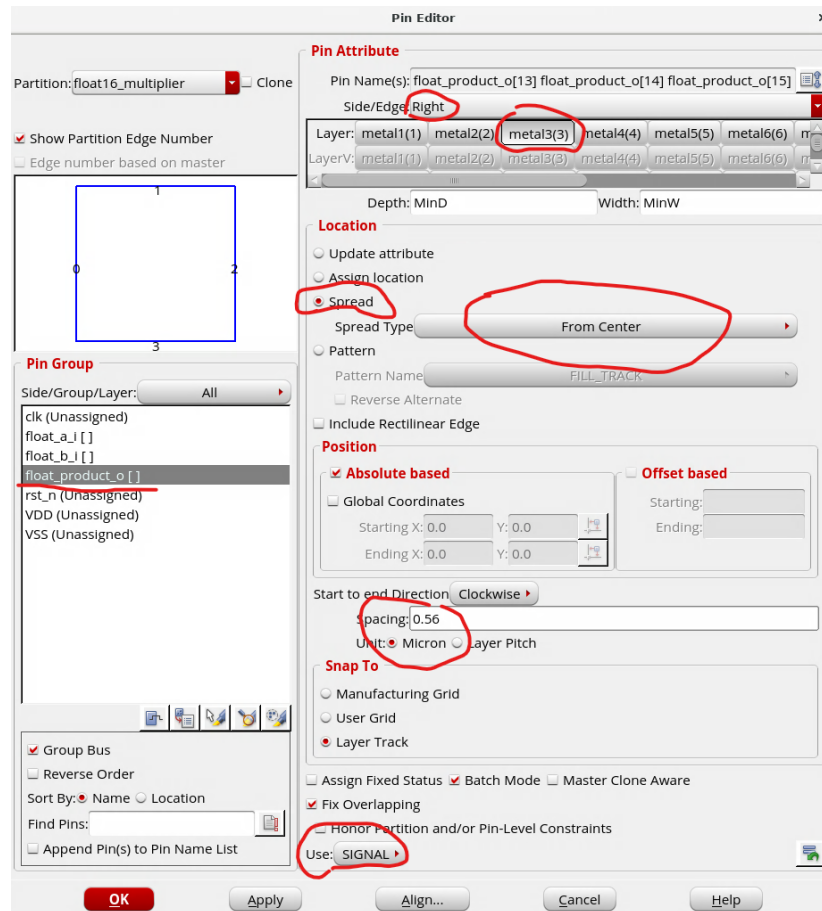
Side/Edge: Right,

Layer: M3 layer,

Spread: from Center,

Spacing: 0.56.

Use: SIGNAL



The input pins, 'clk', 'float_a_i[]', 'float_b_i[]', 'rst_n' can be allocated on the left side with space 0.42.

Side/Edge: Left,

Layer: M3 layer,

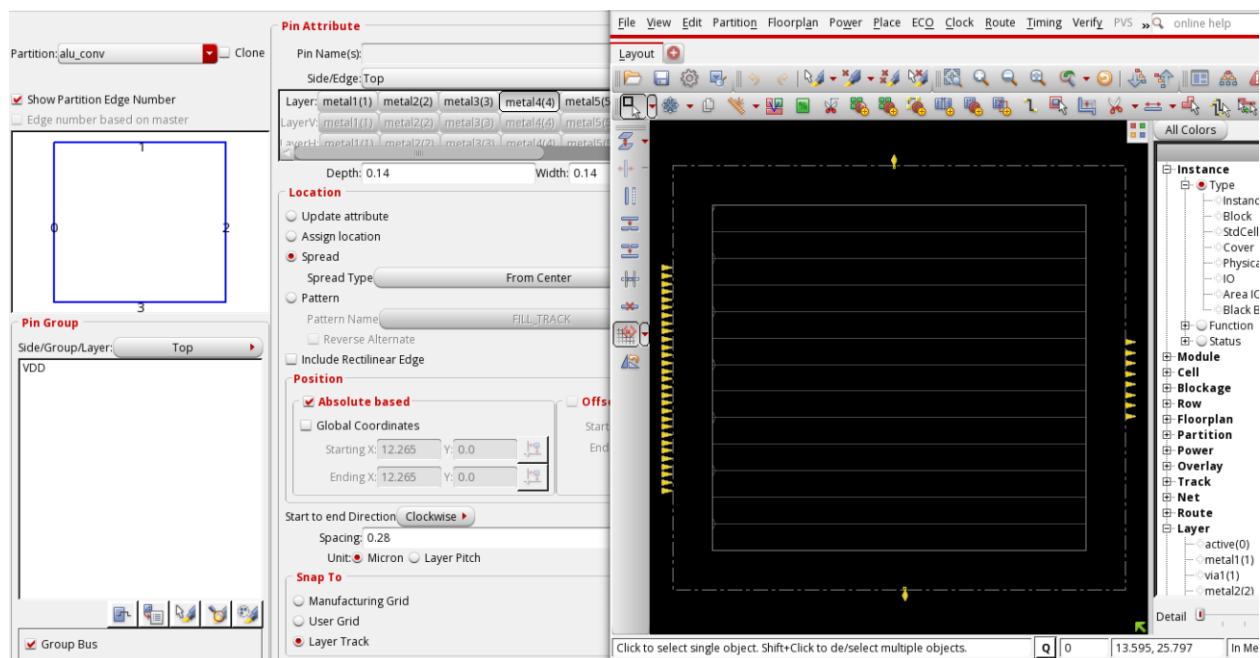
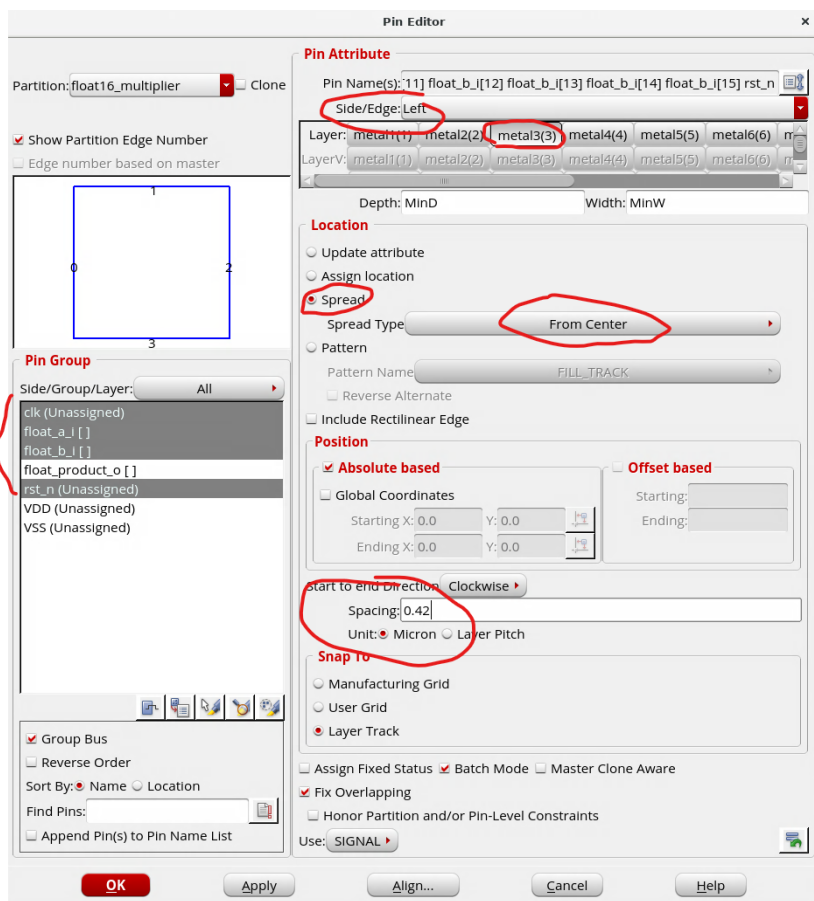
Spread: from Center,

Spacing: 0.42.

Use: SIGNAL

The power and ground pins, VDD, VSS can be allocated on the TOP and Bottom.

The location and space value are not fixed. If all the pins are well allocated and shown, it is also ok.



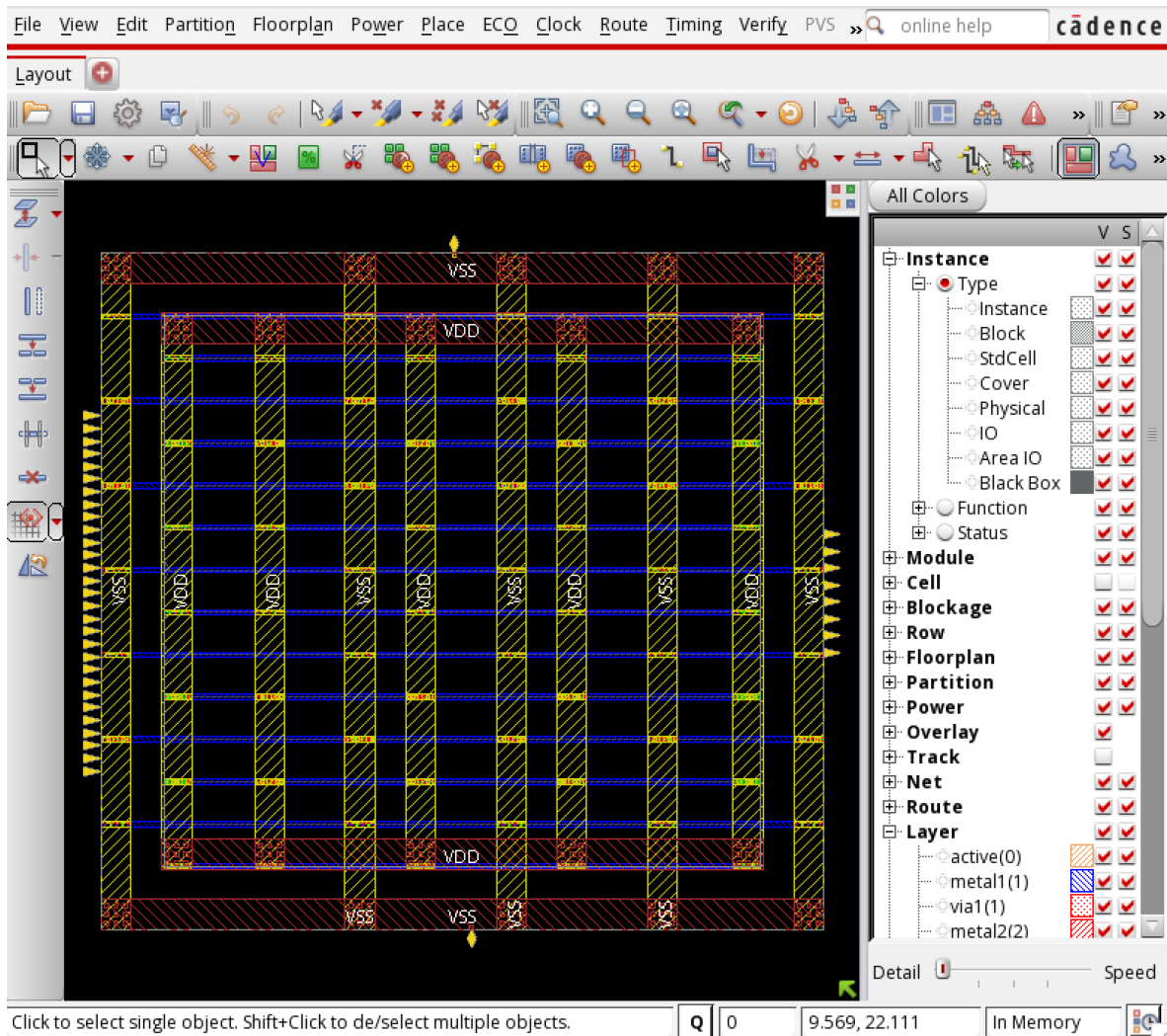
11) \$ saveDesign float16_multiplier_fl.enc

12) # The following command create power ring and stripes.

```
$ addRing -nets {VSS VDD} -type core_rings -follow io -layer {top metal5 bottom metal5 left metal4 right metal4} -width
{top 1 bottom 1 left 1 right 1} -spacing {top 1 bottom 1 left 1 right 1} -offset {top 0 bottom 0 left 0 right 0} -center 0 -
extend_corner {} -threshold 0 -jog_distance 0 -snap_wire_center_to_grid None

$ addStripe -block_ring_top_layer_limit metal5 -max_same_layer_jog_length 1.6 -padcore_ring_bottom_layer_limit metal3
-set_to_set_distance 5 -stacked_via_top_layer metal10 -padcore_ring_top_layer_limit metal5 -spacing 1 -xleft_offset 1 -
merge_stripes_value 0.095 -layer metal4 -block_ring_bottom_layer_limit metal3 -width 1 -nets {VSS VDD} -
stacked_via_bottom_layer metal1

$ srute -connect { blockPin padPin padRing corePin floatingStripe } -layerChangeRange { 1 10 } -blockPinTarget {
nearestRingStripe nearestTarget } -padPinPortConnect { allPort oneGeom } -checkAlignedSecondaryPin 1 -blockPin
useLef -allowJogging 1 -crossoverViaBottomLayer 1 -allowLayerChange 1 -targetViaTopLayer 10 -crossoverViaTopLayer
10 -targetViaBottomLayer 1 -nets { VDD VSS }
```



13) \$ saveDesign float16_multiplier_power.enc

14) # The following command place standard cells

```
$ setEndCapMode -reset
$ setEndCapMode -boundary_tap false
```

```
$ setPlaceMode -reset
```

```
$ setPlaceMode -congEffort auto -timingDriven 1 -modulePlan 1 -clkGateAware 1 -powerDriven 0 -ignoreScan 1 -reorderScan 1 -ignoreSpare 0 -placeIOPins 0 -moduleAwareSpare 0 -preserveRouting 0 -rmAffectedRouting 0 -checkRoute 0 -swapEEQ 0
```

```
$ setPlaceMode -fp false
```

```
$ placeDesign
```

Note: It is fine to have WARNING below, because this design does not include scan chain, which is important in ASIC design. But for this case, it's ok to skip scan chain design.

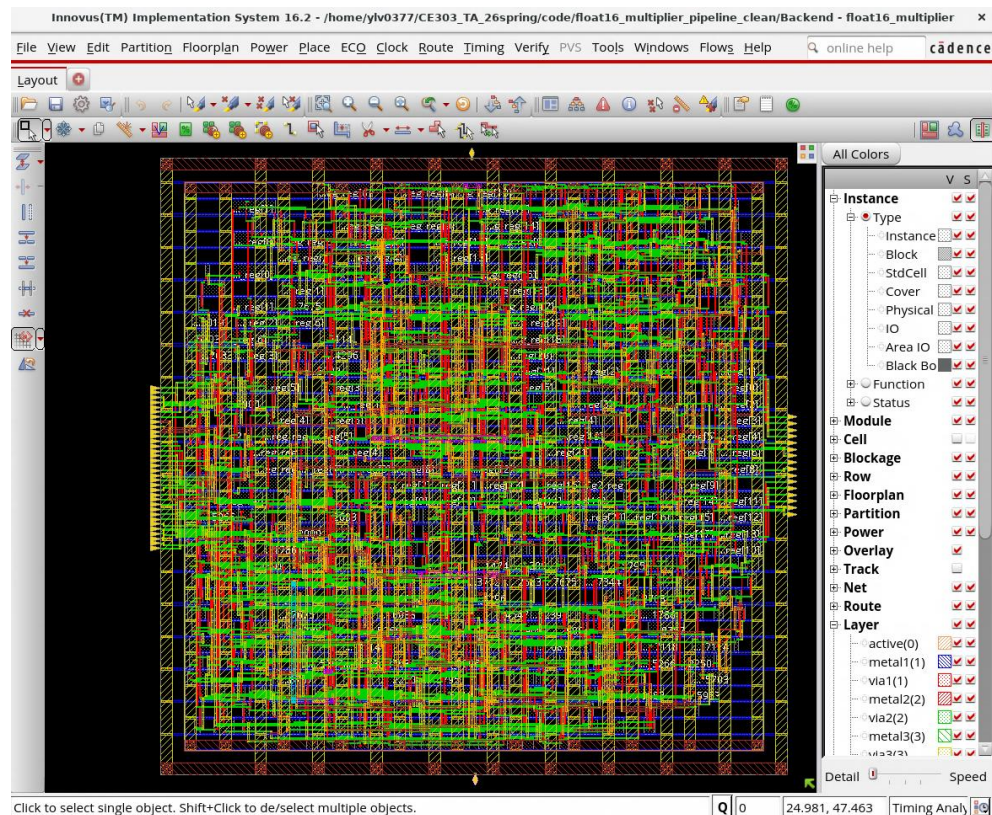
```
*** Summary of all messages that are not suppressed in this session:
Severity ID Count Summary
WARNING IMPDC-1629 1 The default delay limit was set to %d. T
WARNING IMPSP-9025 1 No scan chain specified/traced.
WARNING IMPSP-9042 1 Scan chains were not defined, -place_glo.
*** Message Summary: 3 warning(s), 0 error(s)
```

```
$ timeDesign -preCTS -numPaths 200
```

```
$ optDesign -preCTS -numPaths 200
```

```
$ setDrawView place
```

```
$ saveDesign float16_multiplier_pl.enc
```



15) #The clock system will be established. Detailed clock specification is in clk_gen.ccopt file.

```
$set_ccopt_property update_io_latency false
```

```
$set_ccopt_property post_conditioning_enable_routing_eco 1
```

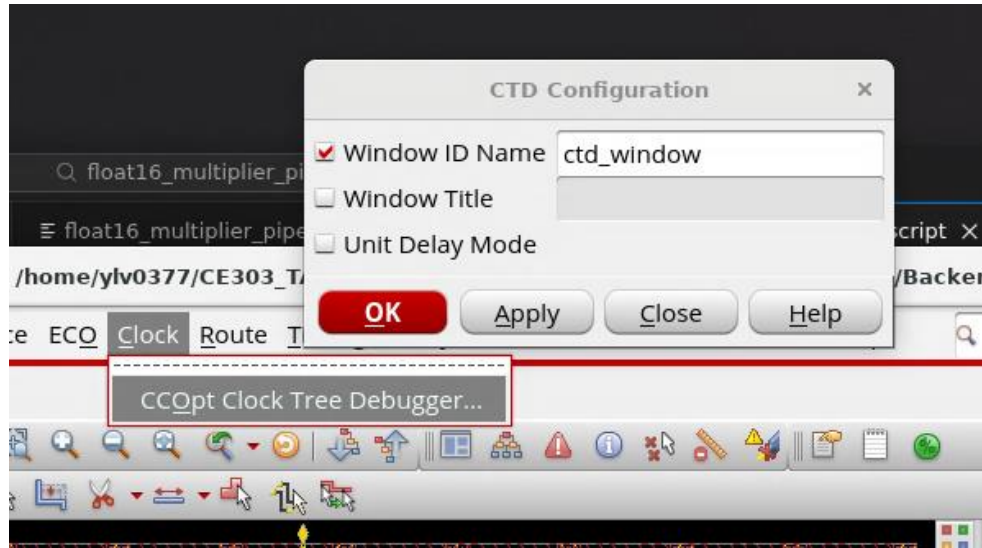
```
$set_ccopt_property -cts_def_lock_clock_sinks_after_routing true
```

```
$setOptMode -unfixClkInstForOpt false
```

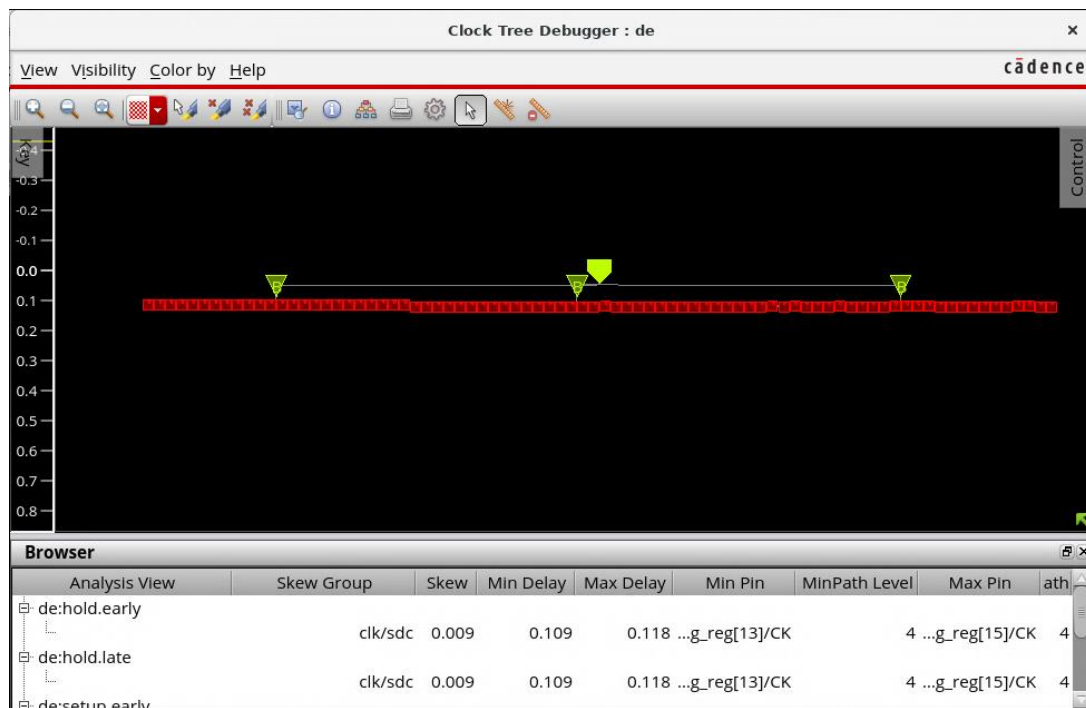
```
$create_ccopt_clock_tree_spec -file clk_gen.ccopt
```

```
$ccopt_design
```

#After running ccopt_design, open the Clock Browser from the Innovus GUI.



click OK and you will see a window similar to the one below, which is the generated clock tree in this design:



\$ checkPlace alu.checkPlace

\$ timeDesign -postCTS -numPaths 200

\$ timeDesign -postCTS -hold -numPaths 200

\$ optDesign -postCTS -numPaths 200

\$ optDesign -postCTS -hold -numPaths 200

\$ timeDesign -postCTS -hold -numPaths 200

\$ timeDesign -postCTS -numPaths 200

```

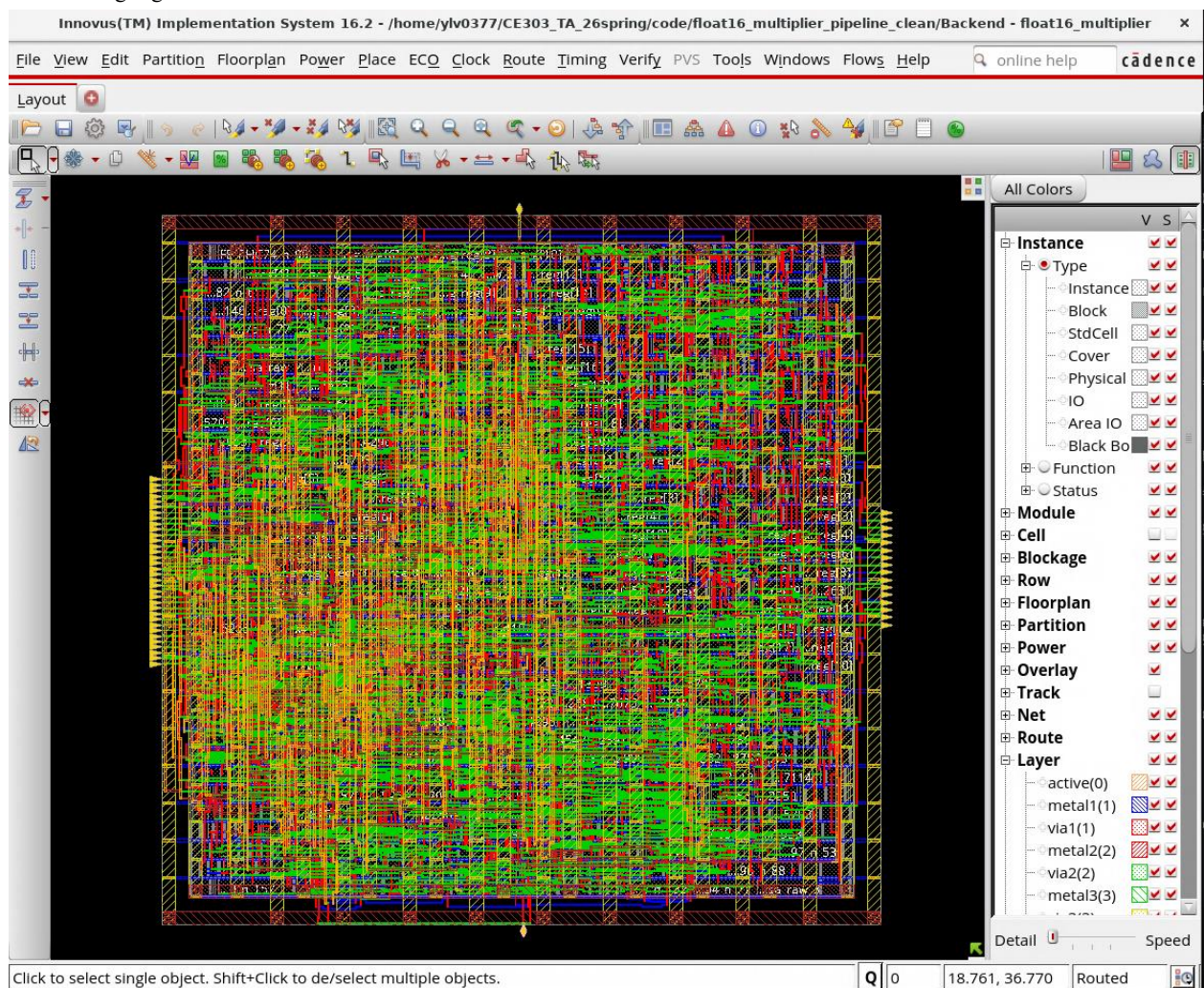
$ saveDesign float16_multiplier_clk.enc

16) # The following command add filler cells to the design
$ getFillerMode -quiet
$ addFillerGap 0.6
$ addFiller -cell FILLCELL_X1 FILLCELL_X2 FILLCELL_X4 FILLCELL_X8 -prefix FILLER -markFixed
$ saveDesign float16_multiplier_powerroute_clk_filler.enc

$ setAnalysisMode -cppr none -clockGatingCheck true -timeBorrowing true -useOutputPinCap true -sequentialConstProp
false -timingSelfLoopsNoSkew false -enableMultipleDriveNet true -clkSrcPath true -warn true -usefulSkew false -
analysisType onChipVariation -log true

$ setNanoRouteMode -quiet -drouteFixAntenna false
$ setNanoRouteMode -quiet -routeTopRoutingLayer default
$ setNanoRouteMode -quiet -routeBottomRoutingLayer default
$ setNanoRouteMode -quiet -drouteEndIteration default
$ setNanoRouteMode -quiet -routeWithTimingDriven false
$ setNanoRouteMode -quiet -routeWithSiDriven false
$ setNanoRouteMode -quiet -routeTopRoutingLayer 6
$ routeDesign -globalDetail

```



```
# Here we remove all the fillers and do the post-route optimization to fix setup/hold violation
$ deleteFiller -prefix FILLER
$ optDesign -postRoute
$ optDesign -postRoute -hold
```

```
$ setFillerMode -add_fillers_with_drc true
$ addFiller -cell FILLCELL_X1 FILLCELL_X2 FILLCELL_X4 FILLCELL_X8 -prefix FILLER -markFixed
$ verify_drc
$ ecoRoute
$ verify_drc
```

17) # Timing analysis are established by the following commands

```
$ timeDesign -postRoute -pathReports -drvReports -slackReports -numPaths 400 -prefix float16_multiplier_postRoute -
outDir timingReports
$ timeDesign -postRoute -hold -pathReports -slackReports -numPaths 400 -prefix float16_multiplier_postRoute -outDir
timingReports
$ saveDesign float16_multiplier_final_layout.enc
```

Note, all timing reports are saved under “timingReports” folder. You can find timing analysis for setup and hold under this folder. You can also type “report_timing” to get the timing results on the screen.

Setup views included:

```
an
```

Setup mode	all	reg2reg	default
WNS (ns):	0.004	0.004	0.008
TNS (ns):	0.000	0.000	0.000
Violating Paths:	0	0	0
All Paths:	200	85	115

Hold views included:

```
an
```

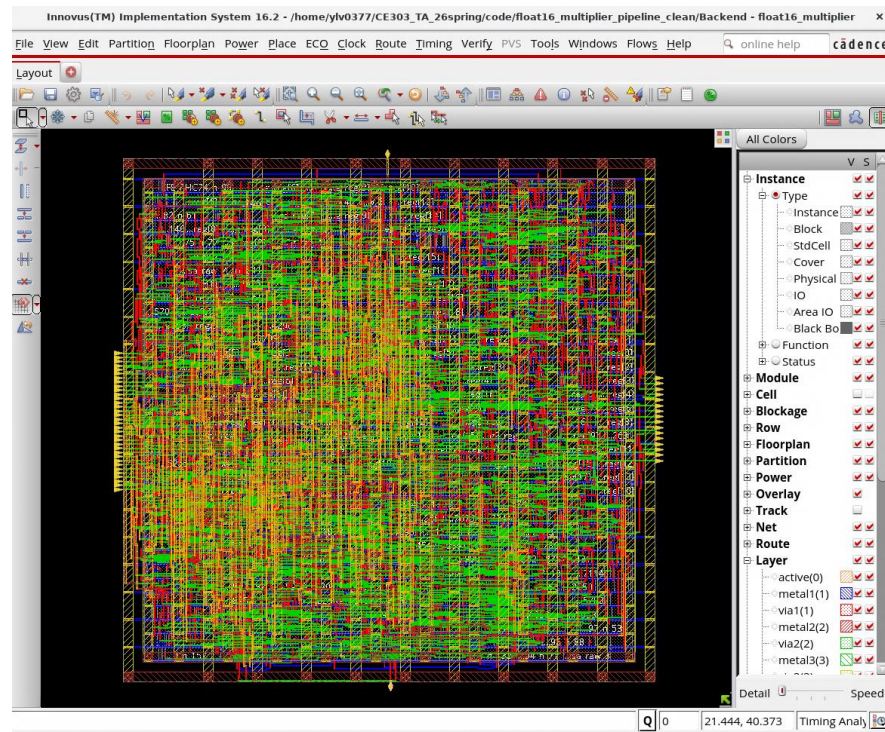
Hold mode	all	reg2reg	default
WNS (ns):	0.001	0.008	0.001
TNS (ns):	0.000	0.000	0.000
Violating Paths:	0	0	0
All Paths:	200	85	115

```
Density: 75.772%
(100.000% with Fillers)
```

You should see zero violating paths, otherwise you may have to change some settings and redo the backend.

18) # Generate “float16_multiplier_phy.v”: used for gate level netlist simulation (Xcelium)

```
$ saveNetlist -phys -includePowerGround float16_multiplier_phy.v -excludeLeafCell
$ saveNetlist float16_multiplier_nophy.v -excludeLeafCell
$ write_sdf float16_multiplier.sdf
```



19) # DRC check, Connectivity check and Antenna check. It is supposed to be 0 violation and 0 warning.

Exit after DRC check, connectivity check and Antenna check.

After checking, you will finish all the Innovus flow operations.

```
$ verify_drc -report float16_multiplier.drc.rpt -limit 1000
```

```
$ verifyConnectivity -type all -error 1000 -warning 50
```

```
$ verifyProcessAntenna -reportfile float16_multiplier.antenna.rpt -error 1000
```

```
$ exit
```

```

innovus 72> verify_drc -report float16_multiplier.drc.rpt -limit 1000
#-report float16_multiplier.drc.rpt # string, default="", user setting
*** Starting Verify DRC (MEM: 1552.3) ***

VERIFY DRC ..... Starting Verification
VERIFY DRC ..... Initializing
VERIFY DRC ..... Deleting Existing Violations
VERIFY DRC ..... Creating Sub-Areas
VERIFY DRC ..... Using new threading
VERIFY DRC ..... Sub-Area : 1 of 1
VERIFY DRC ..... Sub-Area : 1 complete 0 Viols.

Verification Complete : 0 Viols.

*** End Verify DRC (CPU: 0:00:00.2 ELAPSED TIME: 0.00 MEM: 30.8M) ***

innovus 73> verifyConnectivity -type all -error 1000 -warning 50
VERIFY_CONNECTIVITY use new engine.

***** Start: VERIFY CONNECTIVITY *****
Start Time: Tue May 19 00:40:44 2026

Design Name: float16_multiplier
Database Units: 2000
Design Boundary: (0.0000, 0.0000) (53.9600, 53.2000)
Error Limit = 1000; Warning Limit = 50
Check all nets

Begin Summary
  Found no problems or warnings.
End Summary

End Time: Tue May 19 00:40:44 2026
Time Elapsed: 0:00:00.0

***** End: VERIFY CONNECTIVITY *****
Verification Complete : 0 Viols. 0 Wrngs.
(CPU Time: 0:00:00.1 MEM: 0.000M)

innovus 74> verifyProcessAntenna -reportfile float16_multiplier.antenna.rpt -error 1000

***** START VERIFY ANTENNA *****
Report File: float16_multiplier.antenna.rpt
LEF Macro File: float16_multiplier.antenna.lef
Verification Complete: 0 Violations
***** DONE VERIFY ANTENNA *****
(CPU Time: 0:00:00.1 MEM: 0.000M)

```

20) # To verify the final layout is correct, gate level simulation can be run. (use float16_multiplier_nophy.v)

You should already generate two files for the following Xcelium simulations:

1. float16_multiplier_nophy.v (This is the gate level netlist file)
2. float16_multiplier.sdf (This is the timing annotation file for your gate level simulation)

A few steps to setup gate-level simulation with delays:

- a. add one line of code: `timescale 1ns/10ps at the top of float16_multiplier_nophy.v file. Now this file represents all your design
- b. open up float16_multiplier.sdf using vim, and type the command below:

:13,\$s/\///g

Then enter and close the file with command :wq

This will fix some syntax error in the sdf file.

The command above is for “vim” text editor. If you use other text editor, you need to replace “/” with “\” for getting the correct syntax of SDF file for gate simulation.

```

yiv0377@joker:Backend
File Edit View Search Terminal Help
(INTERCONNECT g10645__6083\ZN g10626__4547\A1 (0.001::0.001) (0.001::0
.001))
(INTERCONNECT g10645__6083\ZN g10627__9682\A1 (0.000::0.000) (0.000::0
.000))
(INTERCONNECT g10645__6083\ZN g10628__2683\A2 (0.001::0.001) (0.001::0
.001))
(INTERCONNECT g10645__6083\ZN g10634__7118\A1 (0.000::0.000) (0.000::0
.000))
(INTERCONNECT g10645__6083\ZN g10635__8757\A2 (0.001::0.001) (0.001::0
.001))
(INTERCONNECT g10645__6083\ZN g10636__1786\A1 (0.000::0.000) (0.000::0
.000))
(INTERCONNECT g10645__6083\ZN g10637__5953\A2 (0.001::0.001) (0.001::0
.001))
(INTERCONNECT g10645__6083\ZN g10639__5703\A1 (0.000::0.000) (0.000::0
.000))
(INTERCONNECT g10642__7114\ZN g10640\A (0.000::0.000) (0.000::0.000))
(INTERCONNECT g10643__5266\ZN g10633__7675\A1 (0.000::0.000) (0.000::0
.000))
(INTERCONNECT g10643__5266\ZN g10641\A (0.000::0.000) (0.000::0.000))
(INTERCONNECT g10641\ZN g10603__8757\A2 (0.000::0.000) (0.000::0.000))
(INTERCONNECT g10640\ZN g10632__2391\A (0.000::0.000) (0.000::0.000))
(INTERCONNECT g10639__5703\ZN g10630__6877\B1 (0.000::0.000) (0.000::0
.000))
(INTERCONNECT g10639__5703\ZN g10638\A (0.000::0.000) (0.000::0.000))
(INTERCONNECT g10638\ZN g10629__1309\B1 (0.000::0.000) (0.000::0.000))
:1,$s/\///g

```

c. get back to your testbench file in “Frontend” folder and modify the one line of code in the initial block as below by changing the path of sdf file to your sdf file.

\$sdf_annotate("YOUR SDF FILE PATH", YOUR INSTANCE NAME);

Eg:

```

float16_multiplier dut (
    .clk          (clk),
    .rst_n        (rst_n),
    .float_a_i    (float_a),
    .float_b_i    (float_b),
    .float_product_o(float_product)
);

initial clk = 1'b0;
always #0.5 clk = ~clk; // generate clk

initial begin
// change the sdf file path below to your path of the sdf file
    $sdf_annotate("/vol/ece303/backend_tutorial/Backend/float16_multiplier.sdf", dut);
    clk          = 1'b0;
    rst_n        = 1'b0;
    float_a      = 16'h0000; float_b = 16'h0000;
    #1 rst_n = 1'b1;
    #0.5;

```

This is to add timing information to the instantiated block.

d. prepare float16_multiplier_tb_gatesim.f file under your main folder which contains path to two Verilog files:

one is the float16_multiplier_nophy.v you created during backend, another one is the NangateOpenCellLibrary.v that contains all the cell information

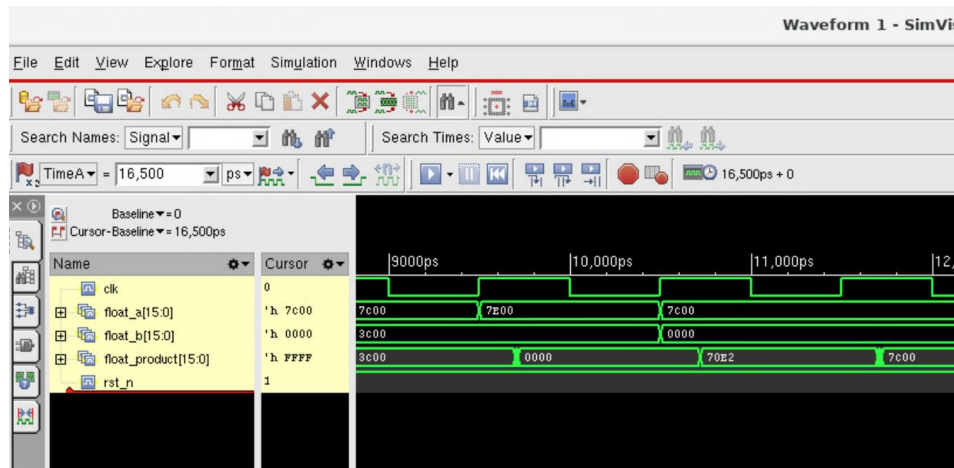
```
/vol/ece303/genus_tutorial/NangateOpenCellLibrary.v
/vol/ece303/backend_tutorial/Backend/float16_multiplier_nophy.v
```

Note: You can use the exact same path for the first file as shown in the screenshot. You need to use your own path of the newly generated “float16_multiplier_nophy.v” for the second file.

Now you should have all the essentials to run back-annotated simulations.

Use the command:

```
# xrun -64bit -gui -notimingcheck -f float16_multiplier_tb_gatesim.f -timescale 1ns/10ps -access rwc -xmclab_args "-sdfstats temp -sdf_verbose -warnmax 0 -delay_mode none -mindelays" float16_multiplier_pipe_tb.v
```



You should see some delays between output and clk.

21) # After the above check, you have finished all the Innovus flow operations.

Below are some additional commands when you have finished the design.

1. Report_area

Enter report_area in Innovus, and it will return the design name, design area, and gate numbers. #Inst is the gate number of your design.

```
innovus 1> report_area
Depth  Name                               #Inst  Area (um^2)
-----
0      float16_multiplier                    1116   1840.986
1
```

2. Report_timing (“report_timing -late” is for setup analysis, “report_timing -early” is for hold analysis; there are many options under report_timing)

Enter report_timing in Innovus, and it will perform static timing analysis on the design.

```

Path 1: MET Setup Check with Pin float_product_reg_reg[6]/CK
Endpoint: float_product_reg_reg[6]/D (v)
checked with leading edge of 'clk'
Beginpoint: product_normalizer_inst_unrounded_product_exp_reg_reg[1]/QN (^)
triggered by leading edge of 'clk'
Path Groups: {clk}
Analysis View: an
Other End Arrival Time 0.118
- Setup 0.031
+ Phase Shift 1.000
- Uncertainty 0.030
= Required Time 1.057
- Arrival Time 1.053
= Slack Time 0.004
Clock Rise Edge 0.000
+ Clock Network Latency (Prop) 0.117
= Beginpoint Arrival Time 0.117

```

3. Power analysis

To report power, you can go to Power -> Power analysis -> Run.

Set to the expected activity factor and clock frequency, OK, then you can observe the design power in the terminal.

